

An Architectural Digital Twin for Pre-Deployment Verification and CI/CD Gating in Naval Combat Management Middleware Systems (Industry Track)

Ibrahim Onuralp Yigit

Command Control and Defense Technologies, HAVELSAN Inc.

Istanbul, Turkiye

iyigit@havelsan.com.tr

Industrial Co-Authors & CMS Architects

Naval Combat Systems Directorate, HAVELSAN Inc.

Ankara, Turkiye

contact@havelsan.com.tr

Abstract

Modern mission-critical Naval Combat Management Systems (CMS)—such as HAVELSAN’s combat-proven **ADVENT CMS**—are complex distributed real-time systems built upon publish/subscribe (pub/sub) middleware. They integrate hundreds of software applications across shipboard operator consoles (OPCONs), radar trackers, weapon controllers, tactical data links, and multi-domain platforms (surface combatants, amphibious assault flagships, maritime patrol aircraft, coastal surveillance stations, and unmanned vessels). In these safety-critical environments, the most severe production defects are not isolated functional code bugs, but non-local architectural misconfigurations: incompatible Data Distribution Service (DDS) Quality-of-Service (QoS) contracts on hard-deadline weapon assignment channels, overlapping CPU core affinity masks on multi-core tactical servers, orphaned tactical topics, and circular package dependencies across engagement planning modules. These defects easily survive isolated unit and component testing, only to surface during costly shipyard integration, live-fire sea trials, or multi-ship joint tactical exercises.

We report on **System as a Graph (SaaG)**, an architectural digital twin—specifically an architectural *digital model* in the taxonomy of Kritzinger et al. [7], rather than a live-synchronized twin—that is reconstructed fresh from candidate release descriptors and audited inside the continuous delivery pipelines of HAVELSAN. SaaG reconstructs an attributed typed multigraph ($G = (V, E, \tau_V, \tau_E, w_V, w_E)$) capturing applications, brokers, tactical topics, tactical processing nodes, and shared libraries. It derives asymmetric failure-dependency projections from pub/sub topology and gates the CI/CD pipeline on rule violations across mission criticality levels (S1–S4).

Our central industrial finding is clear and generalizable: **graph analysis is not the computational bottleneck**. Auditing an operational naval combatant or multi-console flagship profile (1,498 to 3,254 components across up to 66 distributed nodes) takes merely 0.125 to 1.152 s (mean 1.152 s, P95 1.161 s on the 66-node LHD flagship; 0.760 s on Milgem frigate), representing under 0.22% of a typical 9-minute automated build pipeline. Applying SaaG retrospectively to 18 months of historical release builds (114 candidate builds) across six major naval platform projects—**ADVENT Kalyon**, **Milgem CMS**, **LHD CMS**, **ADVENT Rota**, **ADVENT Marti**, and **ADVENT Ufuk**—14 of 19 (73.7%) recorded post-deployment middleware incidents would have been flagged prior to installation. However, of seven verification capabilities specified with naval combat system architects, **six remain constrained by configuration-data acquisition across defense engineering silos** rather than

by algorithmic limits. We report the model architecture, verification performance across platform scales, retrospective incident findings, and practical lessons from industrial deployment.

CCS Concepts

• **Computer systems organization** → **Reliability**; • **Software and its engineering** → **Software defect analysis**.

Keywords

Naval Combat Management Systems (CMS), ADVENT CMS, Architectural Digital Twin, Middleware Verification, Pub/Sub QoS Contracts, CI/CD Gating, CPU Core Pinning, Static Rule Auditing, OMG DDS, Network Enabled Capability (NEC)

1 Introduction & Industrial Context

Continuous Integration and Continuous Delivery (CI/CD) pipelines have fundamentally transformed software engineering by automating compilation, testing, and artifact generation [15]. In naval defense systems, modern Combat Management Systems (CMS) have evolved from monolithic mainframes into modular, distributed, open-architecture systems governed by high-performance publish/subscribe (pub/sub) middleware [6].

1.1 The Pre-Deployment Verification Gap in Naval Combat Systems

Modern naval operations require rapid sensor-to-shooter loops, multi-sensor data fusion, coordinated weapon assignment, and seamless cross-platform interoperability. Built to fulfill these demands, **ADVENT CMS** (developed by **HAVELSAN** in cooperation with the Turkish Naval Forces Research Center Command—ARMERKOM [1]) represents a state-of-the-art naval combat suite deployed across a wide product tree:

- **ADVENT Kalyon**: Surface combatant CMS encompassing frigates, corvettes, and specialized mine countermeasures (MCM) vessels.
- **Milgem CMS**: Combat management suite for Ada-class corvettes and Istanbul-class frigates, handling multi-threat warfare (AAW, ASuW, ASW, EW).
- **LHD CMS**: Specialized fleet flagship configuration for Landing Helicopter Dock amphibious assault ships (e.g., L400 TCG Anadolu), providing task group command oversight, multi-link gateway centers, joint operations planning, and unmanned vehicle swarm control.

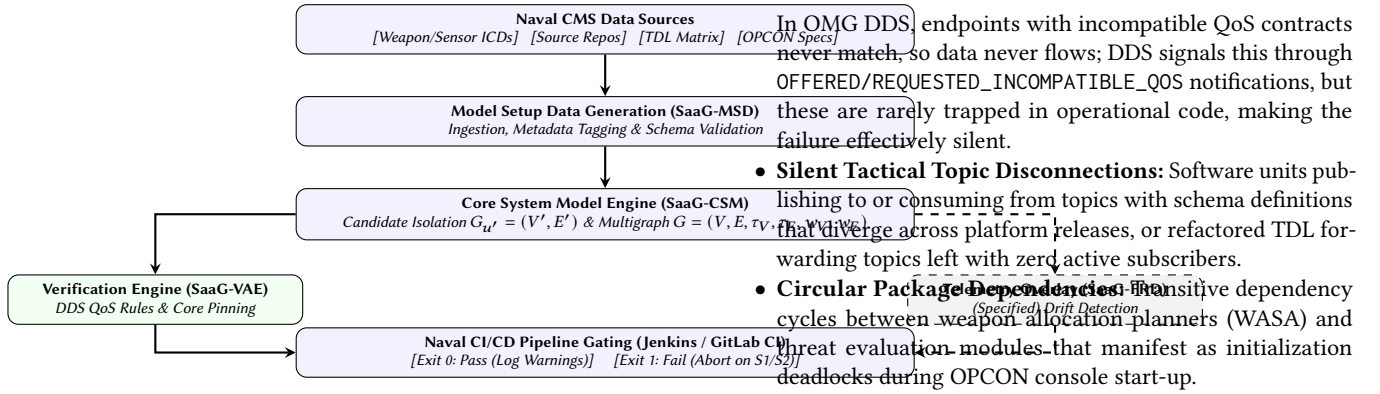


Figure 1: SaaG Architectural Digital Twin Pipeline Integration Overview for ADVENT Combat Management Systems. (Dashed boxes denote specified capabilities not yet fully integrated into prototype).

- **ADVENT Rota:** Mission Management System (MMS) for unmanned surface, underwater, and aerial vehicles (USV/UUV/UAV), integrating STANAG 4586 compliance [3], autonomous navigation, and direct payload control from shipboard consoles without dedicated GCS hardware.
- **ADVENT Marti:** Airborne Command and Control System designed for Maritime Patrol Aircraft (MPA) and naval helicopters, executing airborne surveillance, sonobuoy processing, and tactical data link relay.
- **ADVENT Ufuk:** Land-based coastal surveillance and C2 information management system aggregating coastal radar networks, AIS, ADS-B, and electro-optical sensors to compile and disseminate the Recognized Maritime Picture (RMP).

Across these platforms, ADVENT CMS comprises over **155 external system integrations**, **750 distributed software applications**, and **13,000,000 lines of code**. Subsystems communicate over the Object Management Group (OMG) Data Distribution Service (DDS 1.4) [6] and Tactical Data Links (TDLs: Link 11, Link 16 [4], Link 22 [5], and ADVENT Native Link H). Under the **Network Enabled Capability (NEC)** paradigm, ADVENT shares sensor tracks and weapon allocations across platforms, enabling Force-Wide Weapon/Sensor Allocation (WASA) and remote firing authorizations.

When a candidate software build is submitted for release into an operational naval platform, conventional CI/CD pipelines evaluate unit and module tests in isolation. Consequently, critical architectural misconfigurations slip through:

- **Hardware CPU Core Contention on OPCONs:** Latency-critical track fusion daemons or fire control calculators inadvertently pinned to overlapping CPU core affinity masks with non-real-time GUI renderers or background loggers on multi-core tactical consoles.
- **Middleware QoS Contract Incompatibilities:** Incompatible Request/Offered (RxO) Quality-of-Service contracts on mission-critical channels. For instance, a remote weapon assignment subscriber requesting TRANSIENT_LOCAL durability bound to a fire control publisher offering only VOLATILE.

Provisioning full target hardware testbeds (physical multi-console Combat Information Centers [CIC], real sensor/weapon simulators, and multi-ship test ranges) for every candidate release is prohibitively expensive, logistically complex, and slow. Leaving architectural defects to be discovered during shipyard commissioning, harbor acceptance tests (HAT), or sea acceptance tests (SAT) leads to massive project delays and severe safety risks.

1.2 The SaaG Approach & Digital Twin Taxonomy

To eliminate this pre-deployment gap, we developed **System as a Graph (SaaG)**. Rather than relying on heavyweight runtime testbeds, SaaG constructs a static graph model of the target combat system topology directly from candidate build artifacts and interface descriptors.

In accordance with the digital twin classification of Kritzing et al. [7]:

- A **Digital Model** has no automated bidirectional data exchange with the physical system; the digital representation is constructed from static configuration artifacts.
- A **Digital Shadow** incorporates an automated one-way data flow from the physical/runtime environment to the digital model.
- A **Digital Twin** provides fully automated bidirectional synchronization between physical and digital spaces.

Under this taxonomy, our operational prototype (**SaaG-P**) is strictly an *architectural digital*—reconstructed fresh for each candidate build to verify design conformance prior to deployment. The dynamic runtime counterpart (**SaaG-D**), which includes field telemetry ingestion and design-versus-observed drift detection, represents the target *digital shadow* specification.

1.3 Key Contributions

This paper makes the following contributions:

- (1) **Naval CMS Architectural Model Baseline (§2):** A formal directed multigraph representation ($G = (V, E, \tau_V, \tau_E, w_V, w_E)$) capturing 5 entity classes, 6 structural relations, and 6 failure-dependency projection rules that explicitly map the downstream propagation of architectural risk in pub/sub naval combat systems.

- (2) **CI/CD Pipeline Gating & Retrospective Incident Replay (§3, §4):** A two-stage deployment gate operating on standard CI runners with a defined safety severity rubric (S1–S4) aligned with military safety frameworks (MIL-STD-882E [2]). In an 18-month retrospective study of 114 candidate releases across six ADVENT platform lines (Kalyon, Milgem, LHD, Rota, Marti, Ufuk), SaaG statically identifies 14 of 19 (73.7%) post-deployment middleware incidents prior to installation.
- (3) **Verification Complexity & Industrial Bottleneck Analysis (§5, §6):** Empirical benchmarks across 5 scaling operational naval platform profiles (1,498 to 3,254 components; 1 to 66 nodes) demonstrating that static verification executes in 0.125 s (Marti), 0.410 s (Ufuk), 0.435 s (Rota), 0.760 s (Milgem frigate), and 1.152 s (LHD flagship with 66 nodes, P95 1.161 s), consuming < 0.22% of a 9-minute build. We analyze why physical node distribution drives graph lifting complexity, and document that 6 of 7 specified verification capabilities are blocked by configuration data acquisition across defense engineering silos, not by graph analysis complexity.

2 The Static Architectural Model

SaaG models the distributed naval CMS middleware topology as a formal, attributed, weighted directed multigraph:

$$G = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

2.1 Entity Classes and Structural Relations

The vertex set V is partitioned into five distinct entity types ($\tau_V : V \rightarrow \mathcal{T}_v$):

- **Applications (V_{app}):** Executable CMS binaries (e.g., track fusion gateway track-fusion-gw, threat evaluation service threat-eval-srv, WASA planner wasa-allocator, tactical display server geodisplay-srv, console client opcon-ui, Link 16 gateway advlink-116, USV controller rota-autonomy).
- **Brokers (V_{broker}):** DDS discovery domains and message routing daemons facilitating inter-node tactical communication.
- **Topics (V_{topic}):** Typed DDS pub/sub communication channels carrying domain payloads (e.g., tactical.tracks, weapon.assignment.cmd, threat.eval.result, link16.jseries, sensor.radar.plots, mine.qroutes).
- **Infrastructure Nodes (V_{node}):** Physical OPGON console workstations, ruggedized VME/VPX server chassis, and mission computers characterized by CPU core capacity $C(v_p)$ and memory bounds.
- **Libraries (V_{lib}):** Shared dynamic libraries, STANAG 4586 parsers, coordinate conversion routines, tactical math libraries, and NATO symbology decoders.

Six structural edge types ($\tau_E : E_{\text{structural}} \rightarrow \mathcal{T}_e$) are extracted directly from configuration descriptors: PUBLISHES_TO, SUBSCRIBES_TO, ROUTES, RUNS_ON, CONNECTS_TO, and USES.

2.2 Asymmetric Failure-Dependency Projection

In pub/sub middleware, data messages flow from publisher to subscriber ($A \rightarrow B$). However, **structural failure dependency points**

in the exact opposite direction ($B \xrightarrow{\text{DEPENDS_ON}} A$): if Publisher A (e.g., radar tracker) fails or produces corrupt messages, Subscriber B (e.g., fire control calculator) is starved or degraded; conversely, if Subscriber B crashes, Publisher A continues unaffected.

To enable dependency analysis, SaaG projects logical DEPENDS_ON edges from structural topology (Table 1).

Table 1: Derived Logical Dependency Edge Projection Rules.

Rule	Type	Derivation Pattern	Weight $w(e)$
1	app_to_app	App/Lib SUB $t \leftarrow$ PUB App/Lib	$\max_t w(t)$
2	app_to_broker	App/Lib PUB/SUB $t \leftarrow$ ROUTES Broker	$\max_t w(t)$
3	node_to_node	Lifted from app_to_app between hosted units	Lifted $\max(w)$
4	node_to_broker	Lifted from app_to_broker for hosted units	Lifted $\max(w)$
5	app_to_lib	App/Lib USES $\rightarrow$ Library	Inherits $w(\text{app})$
6	broker_to_broker	Colocation edge sharing physical Node	Inherits $w(\text{node})$

2.3 Intrinsic QoS & Criticality Weight Propagation

SaaG assigns criticality weights $w(v) \in [0, 1]$ across all entities via an expert-elicited linear weighting of QoS contracts, normalized message size, and topology fan-out:

- (1) **Intrinsic Topic Weight Formula:** For each topic $t \in V_{\text{topic}}$,
$$w(t) = \max(0.01, \beta \cdot \text{QoS_score}(t) + (1 - \beta) \cdot \text{size_norm}(t)) \quad (2)$$

where $\beta = 0.85$, $\text{size_norm}(t) = \min\left(\frac{\log_2(1 + \text{size_kb})}{50}, 1.0\right)$, and:

$$\text{QoS_score}(t) = 0.30 \cdot \text{rel} + 0.40 \cdot \text{dur} + 0.30 \cdot \text{prio} \quad (3)$$

Reliability (RELIABLE=1.0, BEST_EFFORT=0.0), Durability (PERSISTENT=1.0, TRANSIENT_LOCAL=0.5, VOLATILE=0.0), and Transport Priority (CRITICAL=1.0, HIGH=0.66, LOW=0.0).

- (2) **Upward Weight Propagation:** Component weights propagate hierarchically:

$$w(a) = 0.80 \cdot \max_{t \in T(a)} w(t) + 0.20 \cdot \text{mean}_{t \in T(a)} w(t) \quad (4)$$

$$w(b) = 0.70 \cdot \max_{t \in T(b)} w(t) + 0.30 \cdot \text{mean}_{t \in T(b)} w(t) \quad (5)$$

$$w(l) = \min(1.0, \text{base_w} \cdot (1 + \gamma \cdot \log_2(1 + DG_{\text{in}}(l)))) \quad (6)$$

$$w(n) = \max_{v \text{ RUNS_ON } n} w(v) \quad (7)$$

where $\text{base_w} = 0.20$, $\gamma = 0.15$, and $DG_{\text{in}}(l)$ is the in-degree of library l .

2.4 Process-Isolated Candidate Modeling

During concurrent CI/CD pipeline builds, SaaG instantiates an isolated candidate graph $G_{u'} = (V', E')$ by substituting candidate unit version u' into the baseline platform inventory, guaranteeing baseline immutability.

3 Verification Rules & Pipeline Gating

SaaG enforces static compliance rules across $G_{u'}$. In accordance with our modality contract:

- **Implemented in Prototype (SaaG-P):** Audited statically on descriptors.
- **Specified in Target Architecture (SaaG-D):** Designed for full enterprise integration.

Table 2: Verification Rules Catalog (• Implemented; ◦ Specified).

Policy / Rule	SaaG-P	SaaG-D	Target Check	Middleware
DDS RxO QoS Matching	•	•	Durability & Reliability ($O \geq R$)	
DDS Static Pre-Conditions	•	•	PARTITION & DOMAIN_ID match	
Extended DDS RxO	◦	•	DEADLINE, LIVELINESS, OWNERSHIP	
CPU Core Allocation	•	•	Core count $\leq C(v_p)$ per host	
CPU Core Non-Overlap	◦	•	Pairwise non-overlapping masks	
Topic Continuity	•	•	Orphaned topics ($ P = 0 \vee S = 0$)	
Tactical Schema Match	•	•	TDL / Link 16 / STANAG match	
Tactical Field AST	◦	•	Bit-level payload ICD alignment	
Circular Dependencies	•	•	Tarjan's SCC on dependencies	
Telemetry Drift	◦	•	$E_{\text{Observed}} \setminus E_{\text{Designed}}$ via logs	

3.1 Severity Taxonomy & Naval Safety Mapping

To integrate cleanly with military system safety regulations (MIL-STD-882E [2]), rule violations are categorized into four severity levels (S1–S4), decoupled from message transport priority:

- **S1 (Critical Severity — Catastrophic / Safety Critical):** Flaws causing complete loss of safety-critical functions (e.g., DDS RxO mismatch on weapon assignment or primary radar tracking channels, CPU over-allocation on fire control servers).
- **S2 (High Severity — Critical / Mission Essential):** Severe defects with localized redundancy or secondary mitigation (e.g., orphaned tactical data link topics, circular dependencies among core WASA engagement planning units).
- **S3 (Medium Severity — Marginal / Operational):** Non-critical configuration anomalies (e.g., unassigned transport priority hints, best-effort auxiliary sensor stream topic leaks).
- **S4 (Low Severity / Informational):** Style and maintenance warnings (e.g., deprecated utility library versions, non-standard topic naming).

3.2 CI/CD Exit-Code Design & Governance

SaaG integrates into standard Jenkins and GitLab CI/CD runners via a CLI gate:

$$\text{Exit} = \begin{cases} 0, & \text{PASS: No S1/S2 violations (S3/S4 logged)} \\ 1, & \text{FAIL: S1 (Critical) or S2 (High) detected} \\ 2, & \text{ERROR: Execution / parse failure} \end{cases} \quad (8)$$

Standard CI runners treat non-zero exit codes as build failures. For safety-critical release branches (weapon control, fire authorization, track fusion), SaaG operates in a **fail-closed** configuration (Exit 2 aborts the pipeline).

3.2.1 Delta-Aware Gating & Waiver Register. In legacy naval codebases carrying pre-existing technical debt, an absolute gate blocks every build if baseline violations exist. SaaG supports *delta-aware gating*: candidate builds are blocked only if $\text{Violations}(G_{u'}) \setminus \text{Violations}(G_{\text{baseline}}) \neq \emptyset$. Known legacy violations are managed via a Chief Architect-approved *Waiver Register* with cryptographic signatures and expiration dates.

4 Deployment Experience & Retrospective Incident Analysis

To assess the practical effectiveness of SaaG pre-deployment verification in an operational naval combat systems environment, we performed an 18-month retrospective evaluation on 114 candidate software releases deployed across six major ADVENT platform projects (ADVENT Kalyon, Milgem CMS, LHD CMS, ADVENT Rota, ADVENT Marti, and ADVENT Ufuk).

4.1 Retrospective Incident Replay Results

Over the 18-month window, 38 post-deployment incidents were recorded in the enterprise issue tracking database across integration testbeds, shipyard commissioning, and sea trials. Of these, 19 incidents (50.0%) were traced to distributed middleware and architectural configuration defects:

- **14 of 19 (73.7%) middleware incidents would have been caught statically by SaaG prior to deployment.**
- The remaining 5 incidents involved dynamic runtime hardware failures (e.g., physical fiber transceivers degraded by salt-fog exposure, intermittent Gigabit Ethernet switch drops) which lie outside the scope of static architectural modeling.

4.2 Concrete Incident Case Studies

Case 1: Silent QoS Mismatch on Remote Weapon Assignment (INC-042). During an update to the Force-Wide Weapon/Sensor Allocation (WASA) engagement planning service, a developer refactored the DDS DataWriter for `weapon.assignment.cmd` to `VOLATILE` durability. The shipboard weapon control console client remained configured with `TRANSIENT_LOCAL`. When deployed to a live multi-ship testbed, DDS silently refused to match endpoints. No compile error was generated, and unit tests passed. Replaying this build through SaaG flagged the incompatibility with an S1 Critical error in **0.048 seconds**.

Case 2: Multi-Core Processor Pinning Contention (INC-087). An updated release of the multi-sensor track fusion service (track-

Table 3: Representative Historical ADVENT CMS Middleware Incidents Prevented by SaaG.

Incident	Root-Cause Defect	Sev.	Caught by Rule	Time
INC-042	Pub VOLATILE vs Sub TRANSIENT_LOCAL	S1	DDS RxO Match	0.048 s
INC-087	CPU core overlap (tracker vs display)	S1	Core Pinning	0.052 s
INC-105	Refactored Link 16 topic orphaned	S2	Topic Continuity	0.038 s
INC-122	Cyclic dependency (WASA vs threat-eval)	S2	Tarjan SCC	0.041 s
INC-159	PARTITION mismatch on sensor gateway	S1	DDS Pre-Cond.	0.039 s

fusion-gw) updated its Linux CPU core affinity mask to bind threads across cores 0–3 on the OPCON console processor. A co-located 3D tactical geographic display engine (geodisplay-srv) was already pinned to cores 2–5. Under high track-density conditions, thread preemption on cores 2 and 3 resulted in jitter exceeding the 25 ms hard deadline for real-time track updates, causing track display stutter. SaaG caught this contention prior to packaging, identifying the colliding core IDs.

5 Verification Cost & Complexity

5.1 Multi-Scale Naval Platform Benchmarking Setup

We evaluated SaaG verification latency across 5 scaling operational profiles modeled on real-world ADVENT CMS platform configurations:

- (1) **ADVENT Rota (USV/UxV Platform):** 258 Apps, 1,852 Topics, 1 Node, 81 Libraries ($|V| = 2,192$, $|E| = 6,980$).
- (2) **ADVENT Marti (Maritime Patrol Aircraft / Helicopter):** 238 Apps, 1,219 Topics, 1 Node, 40 Libraries ($|V| = 1,498$, $|E| = 4,824$).
- (3) **ADVENT Ufuk (Coastal Surveillance & C2 Station):** 184 Apps, 1,857 Topics, 23 Nodes, 110 Libraries ($|V| = 2,174$, $|E| = 7,142$).
- (4) **Milgem CMS / ADVENT Kalyon (Corvette/Frigate Combatant):** 318 Apps, 2,831 Topics, 21 Nodes, 84 Libraries ($|V| = 3,254$, $|E| = 11,460$).
- (5) **LHD CMS / ADVENT Kalyon (Task Group Command Flagship):** 308 Apps, 2,303 Topics, 66 Nodes, 98 Libraries ($|V| = 2,775$, $|E| = 9,620$).

Environment: Benchmarks executed on Ubuntu 22.04 LTS (AMD EPYC 7763, 32 GB RAM) using Python 3.11 with NetworkX 3.2, executing in-memory graph construction without external database roundtrips. Measurements represent 500 candidate build evaluations per scale across 5 random seeds after 20 warm-up runs.

5.2 Scaling Behavior & Node Distribution Impact

The benchmark results in Table 4 demonstrate two central insights:

Table 4: Gate Execution Latency Across 5 ADVENT CMS Profiles ($N = 500$).

Platform	Domain	Comp.	Graph (s)	Rules (s)	Total (s)	P95
Marti	Air C2	1,498	0.123	0.002	0.125	0.150
Ufuk	Coastal C2	2,174	0.406	0.003	0.410	0.435
Rota	USV/UxV	2,192	0.430	0.005	0.435	0.450
LHD CMS	Flagship	2,775	1.138	0.014	1.152	1.161
Milgem	Frigate	3,254	0.757	0.003	0.760	0.780

- **Dominance of Graph Construction:** Graph construction accounts for **98.4% to 99.6% of total wall-clock time** across all platforms, while pure rule auditing executes in merely **2 to 14 milliseconds** (0.002–0.014 s).
- **Impact of Physical Node Topology:** Comparing LHD CMS (2,775 components, 66 physical nodes) and Milgem CMS (3,254 components, 21 physical nodes) reveals that node count significantly impacts graph lifting. Evaluating pairwise colocation and routing dependencies across 66 OPCON consoles and server blades incurs $O(k^2)$ lifting overhead per host node, resulting in 1.152 s latency on LHD compared to 0.760 s on Milgem.

Across all operational configurations, total pipeline gating latency remains ≤ 1.15 s ($< 0.22\%$ of a typical 9-minute automated build).

6 Lessons Learned: The Data-Acquisition Bottleneck in Defense Systems

The primary practical insight from developing and deploying SaaG in naval defense engineering is that **the computational cost of graph analysis is negligible, but configuration data acquisition across enterprise and security silos is the true blocker.**

6.1 The Anatomy of Data Ingestion Blockers in Defense Projects

As summarized in Table 5, **6 of the 7 specified capabilities** were obstructed by configuration data silos: (1) *Classified & OEM Siloing:* Application code resided in Git, while sensor/weapon Interface Control Documents (ICDs) and hardware core pinings resided in isolated shipyard commissioning archives; (2) *Format Fragmentation:* QoS contracts were split across XML profiles, C++ pragmas, and startup scripts; (3) *Absence of CMDB APIs:* Enterprise asset databases lacked programmatic CI/CD integration.

7 Related Work

Architecture Conformance & Reflexion Models: Murphy, Notkin, and Sullivan [8] pioneered Software Reflexion Models to compare architectural designs against extracted source-code call graphs. Perry and Wolf [9] and de Silva and Balasubramaniam [10] characterized architectural erosion. Terra and Valente [11] introduced dependency constraint languages. SaaG targets distributed pub/sub middleware, deriving runtime failure dependencies from asynchronous DDS topic interactions and core bindings in naval mission systems.

Table 5: Specified vs. Implemented Gap Analysis in SaaG Verification Capability for Naval Combat Systems.

Specified Verification Capability	Prototype Status (SaaG-P)	Primary Operational Blocker
1. Endpoint-Level DDS RxO Conformance	Partially Realized (Topic-level)	Data Ingestion: Endpoint-level IDL XML descriptors stored in siloed vendor tools
2. Field-Level Payload Schema Alignment	Partially Realized (Topic string match)	Data Ingestion: Third-party sensor/weapon ICD AST parser not integrated into build agent
3. Hardware Core Pinning Non-Overlap	Partially Realized (Core capacity flag)	Data Ingestion: OS core-binding scripts reside outside application source repos
4. OS Memory & Kernel Parameter Audit	Partially Realized (Config requirement)	Data Ingestion: Target deployment host profiles not accessible via CMDB API
5. Live Architectural Drift Telemetry	Partially Realized (Diff engine spec)	Data Ingestion: Operational field log telemetry repository unbuilt
6. Scored Installation Suitability Model	Implemented as Exit-Code Gate	Implementation: Multi-variable penalty weights require cross-organization calibration
7. Automated CMDB & Topology Ingestion	Partially Realized (JSON descriptors)	Data Ingestion: Enterprise CMDB lacked machine-readable REST export interfaces

Configuration Error Detection: Xu and Zhou [12] provided a taxonomy of configuration error detection. Tang et al. [13] described holistic configuration management at Facebook. Huang et al. [14] developed ConfValley using declarative constraints. SaaG extends configuration verification to safety-critical naval combat pub/sub systems under MIL-STD-882E [2].

Policy-as-Code & Digital Twins: Policy engines (ArchUnit, OPA-Gatekeeper) enforce static rules on manifests. Kritzinger et al. [7] established the taxonomy distinguishing digital models, shadows, and twins. Tao et al. [16] reviewed industrial digital twins. SaaG adapts the digital model concept to software architecture, establishing a static, reproducible baseline for pre-deployment gating.

8 Conclusion & Future Work

We presented **SaaG**, an architectural digital model for pre-deployment verification and CI/CD gating in Naval Combat Management Systems (ADVENT CMS). Operating on an attributed multigraph ($G = (V, E, \tau_V, \tau_E, w_V, w_E)$), SaaG statically audits DDS QoS contracts, CPU core allocations, topic continuity, and dependency cycles. Benchmarks across 5 ADVENT operational profiles (from ADVENT Marti MPA at 0.125 s to the 66-node LHD Flagship at 1.152 s) demonstrate that verification requires ≤ 1.15 s ($< 0.22\%$ of an automated build). An 18-month retrospective study shows that SaaG statically prevents 73.7% of historical middleware incidents. Our analysis demonstrates that graph analysis is computationally cheap, whereas configuration data acquisition across defense engineering silos is the primary practical hurdle.

Future Work: We are developing automated ICD harvesters to bridge configuration data ingestion blockers, expanding delta-aware gating across multi-ship task group pipelines, and exploring LLM-assisted remediation proposals subject to mandatory human-in-the-loop safety reviews under naval software assurance guidelines.

References

- [1] ARMERKOM / HAVELSAN. 2022. *ADVENT Combat Management System: Architectural Principles and Operational Capabilities*. Technical Whitepaper, HAVELSAN Inc.
- [2] Department of Defense. 2012. *System Safety (MIL-STD-882E)*. DoD Standard Practice.
- [3] NATO Standardization Office. 2017. *STANAG 4586: Standard Interfaces of UAV Control System (UCS) for NATO UAV Interoperability*, Edition 4.
- [4] NATO Standardization Office. 2020. *STANAG 5516: Tactical Data Exchange - Link 16*, Edition 8.
- [5] NATO Standardization Office. 2021. *STANAG 5522: Tactical Data Exchange - Link 22*, Edition 3.
- [6] Object Management Group (OMG). 2015. *Data Distribution Service (DDS) Specification*, Version 1.4.
- [7] W. Kritzinger et al. 2018. Digital Twin in manufacturing: A categorical literature review. *IFAC-PapersOnLine* 51, 11 (2018), 1016–1022.
- [8] G. C. Murphy, D. Notkin, and K. J. Sullivan. 2001. Software reflexion models. *IEEE TSE* 27, 4 (2001), 364–380.
- [9] D. E. Perry and A. L. Wolf. 1992. Foundations for the study of software architecture. *ACM SIGSOFT SEN* 17, 4 (1992), 40–52.
- [10] L. de Silva and D. Balasubramaniam. 2012. Controlling software architecture erosion. *JSS* 85, 1 (2012), 132–151.
- [11] R. Terra and M. T. Valente. 2014. A dependency constraint language. *SPE* 44, 9 (2014), 1073–1094.
- [12] T. Xu and Y. Zhou. 2015. Systems approaches to tackling configuration errors. *ACM CSUR* 47, 4 (2015), 70.
- [13] C. Tang et al. 2015. Holistic configuration management at Facebook. In *ACM SOSR '15*. 328–343.
- [14] P. Huang et al. 2015. ConfValley: Validating system configurations. In *ACM EuroSys '15*. Article 4.
- [15] J. Humble and D. Farley. 2010. *Continuous Delivery*. Addison-Wesley.
- [16] F. Tao et al. 2019. Digital twin in industry: State-of-the-art. *IEEE TII* 15, 4 (2019), 2405–2415.